SDC Hackathon 2026 — Phase 1 Project Proposal

Paste into the official Submission Template. Save as `SDC2026_<TeamName>_Preliminary.pdf`. Deadline Sat 25 Oct 2026, 07:59:59 MYT. Items in [brackets] are yours to fill.

1. Team and project overview

Field	Value
Team Name	[Team name]
Selected Domain	Domain 2 — Orbyte Smart SME Challenge: Less Busywork, Better Business (Alpha Red Solutions Sdn Bhd)
Selected SDG Track (1–17)	SDG 8 (primary); SDG 12 (secondary)
Project / Solution Name	SupplySwipe (agent name: Wakil)
Name (Leader)	[Leader name]
Email & Phone (Leader)	[email] / [phone]
University Affiliation(s)	[Monash University Malaysia; others]
Number of Team Members	4

2.1 The problem context (≤250 words)

Malaysia's 1.09 million MSMEs include tens of thousands of cafés, restaurants and stalls that buy fresh ingredients every week from small suppliers. Prices move constantly: Grade C eggs rose 7.3% in a single quarter of 2026 and standard chicken climbed to RM9.70/kg as farms closed. So owners renegotiate with three to eight suppliers almost every week, and they do it on WhatsApp: a price question, a counter, a delivery question, a phone call, then an invoice that does not match the chat. Finding a new supplier is worse, because every candidate means the same rounds of messages about price, quantity, delivery and payment terms. Small suppliers face the mirror image, repeating the same quote to dozens of leads and chasing confirmations. Terms agreed in scattered chats are easy to misquote: roughly 39% of manually handled invoices contain errors, and invoice discrepancies are the most common supplier complaint in comparable grocery supply chains. The cost is hours of messaging, lost deals, disputes and over-ordering that ends as waste. Large buyers have already solved this. Walmart lets AI negotiate with suppliers and 82% of those suppliers preferred it; Alibaba's Accio negotiates sourcing but requires a human to approve every price and order. None of this reaches a 40-seat café and a poultry wholesaler in Shah Alam. They have no buying power, no procurement staff and no system of record, only a phone. The burden is not ordering; it is negotiating and remembering what was agreed.

2.2 The proposed AI solution (≤300 words)

SupplySwipe is an Orbyte module in which each business has a private AI agent, its Wakil, that negotiates supply deals while owners approve instead of chasing. Buyers post a request (item, quantity, delivery days,

public target, sealed maximum); suppliers list items (list price, sealed floor, MOQ, capacity, areas, terms). A rule-based feed, with no AI, applies hard filters (halal, category, delivery area, quantity versus capacity) and ranks counterparts with "why this match" chips. A swipe right is the trigger. Both Wakils load their own sealed limits, style and memories of past dealings, plus the request, the listing and the PriceCatcher market price, and exchange at most ten structured messages: ask, answer, offer, counter, accept or walk away. Code checks every message before it is sent: offers outside the agent's own limits, drafts that leak a sealed value, and terms not in the request or listing are rejected and replaced by a rule-based move. The language model is used only where it adds value: reading free-text specs, asking clarifying questions and writing plain-language reasons. A deal is only a proposal: both owners receive one notification and approve, counter through their Wakil, or reject; staff may approve only below an owner-set limit. Approval creates one deal record with pinned terms, opens a chat that cannot change those terms, and writes every step, including blocked drafts, to an append-only activity log. When the supplier's invoice arrives it is compared line by line with the pinned terms, and any mismatch is flagged before payment. Each Wakil then saves a short memory, so next week's renegotiation is faster. From the SDC workshops we use Replit for build and hosting, a GenAI API with structured outputs, and Supabase Postgres with row-level security. The negotiation engine is exposed as a reusable API for other Orbyte modules.

3.1 Conceptual tech stack

Layer	Choice	Why
Frontend	Next.js 14 + TypeScript + Tailwind; Recharts for the price chart; three.js (optional 3D agent with 2D fallback)	Phone-first screens, live negotiation replay, swipe deck
Backend	Python / FastAPI (negotiation engine, validator, feed ranker)	Rules and validation in code; LLM only for wording
Database	Supabase Postgres with row-level security; Realtime for live messages	Sealed limits readable only by their own business; append-only log
LLM	[provider + exact model], structured JSON output only, per-message and per-negotiation token caps	Wording, clarifying questions, reasons; never prices
Hosting	**Replit** (app deployed from the team's own account; deployment URL in the declaration)	Compulsory sponsor platform
Data	Synthetic businesses, requests and listings; market reference modelled on DOSM PriceCatcher (CC BY 4.0)	No real personal or company data
Repo	github.com/kaylaelishevaa/supply-swipe, public, Apache-2.0	Judges can read every component

System block diagram (attach as appendix): Orbyte hub (simulated profiles and stock) and the web app (buyer and supplier views) → Backend API on Replit: feed ranker (code), negotiation engine + validator (two agents, ≤10 JSON messages), dual approval → deal record → activity log → Supabase Postgres (sealed limits, requests, listings, negotiations, approvals, log) and the GenAI API (structured JSON only).

Did your project utilise any official SDC sponsor-provided GenAI API or Cloud Platform? Yes. Replit Core (sponsor-provided) builds and hosts the application; proof from our own account is in the Tech & Tools Declaration. RunPod is not used because no GPU work is needed. The GenAI API is [sponsor-provided / our own account].

3.2 Design-thinking & deployment strategy (≤200 words)

We began by asking café owners to show us their supplier chats and counting the messages it took to agree one deal. Every chat followed the same loop: price, counter, delivery, silence, call, mismatched invoice. That reframed the problem from "ordering" to "negotiating and remembering what was agreed". We rejected a chatbot, because the owner would still be haggling, just with software, and we rejected full autonomy, because owners will not let software commit their money. The design principle became bounded autonomy: owners set limits once, agents work inside them, and a person approves every deal. Supplier interviews made it two-sided: they repeat the same quote to dozens of leads and want a fast, fair, recorded process. Viability comes from putting rules where rules belong, with ranking and validation in code and the model used only for language, which keeps cost and errors low. Usability comes from swipe cards, one-tap approval and plain-language reasons, so a first-time user completes the journey unaided. For Phase 2 we deploy on Replit, test with five outlets and two suppliers on synthetic data, measure messages per agreed deal and limit breaches, and iterate the limits form.

4. Preliminary environmental & SDG impact framework

Metric category	Response
Target SDG sub-target(s)	**Target 8.3** (support MSME productivity, entrepreneurship, formalisation and growth). Secondary **8.2** (productivity through technological upgrading) and **12.3** (reduce food loss along supply chains: quantities negotiated against actual need, invoices checked against agreed terms).
Current problem baseline	Owners exchange an average of **[N] messages over [M] days** to agree terms with one supplier, and **[P]%** had an invoice that did not match agreed terms last month (team survey of [5] F&B owners, Klang Valley, Oct 2026). Context: 1,086,386 MSMEs (DOSM 2024); Grade C eggs +7.3% and chicken +3.3% to RM9.70/kg in Q1 2026 (Ministry of Economy); ~39% of manually processed invoices contain errors (Resolve Pay via NetSuite); 73% of Malaysian SMEs report late payment (SME Media Group/INTI, 2017).
Core measurable KPI	(1) Owner messages and minutes from request to agreed terms per deal. (2) Share of deals captured in one structured record with pinned terms. (3) Agent limit breaches per 50 test negotiations (validator log). (4) Invoice mismatches caught before payment per 100 invoices.
Phase 2 impact target	**≥90% fewer owner messages per agreed deal** (survey baseline → one approval); request-to-approved deal **under 5 minutes**; **100%** of deals within owner limits with **0 breaches** in 50 test negotiations; **100%** of invoices compared to pinned terms before payment; 3 of 3 first-time users complete the journey unaided.

Sources to cite: DOSM MSME statistics 2024 · Business Today, 28 Apr 2026 (egg prices) · NetSuite, invoice disputes (Resolve Pay) · SME.asia late-payment survey · Pactum client results · Digital Commerce 360, 24 Jul 2026 (Accio).

5. Eligibility & readiness checklist

- ☐ All members completed the GenAI, Database and Cloud Deployment workshops *(confirm; recordings on Discord)*
- ☑ Verified real-world problem mapped to UN SDGs
- ☑ Functional, deployed application, not a case study
- ☑ All fields within word limits

6. IP & originality declaration

6.1 Pre-existing work: No. Repository created after the 19 Oct 2026 release; all code, design and research produced during the hackathon.

6.2 Third-party code, datasets, models: Yes.

Name	Type	License
Next.js 14.2 / React 18 / TypeScript	code	MIT
Tailwind CSS 3.4	code	MIT
Recharts 3.10	code	MIT
three.js r149 / r160 (CDN, optional)	code	MIT
Plus Jakarta Sans, JetBrains Mono (Google Fonts)	fonts	SIL OFL
FastAPI, Supabase client libraries (backend)	code	MIT
[LLM provider + model]	model (pre-trained, API)	provider terms
DOSM PriceCatcher (format reference)	dataset (public)	CC BY 4.0
Synthetic businesses, requests, listings, transcript	dataset (team-created)	n/a

6.3 Employment-related IP conflicts: No. *(Each member confirms.)*

7. Declaration

[Leader signature] · [Date]